Generating Function-Informed Initialization for Genetic Algorithms on the Multidimensional Knapsack Problem

Muhammad Marzouk Baig
Northeastern University
Portland, Maine
baig.muham@northeastern.edu

1 Problem Statement

1.1 The Classical 0-1 Knapsack Problem

Given n items each with value v_j and weight w_j, and a knapsack with capacity W, the classical 0-1 knapsack problem is:

$$\begin{aligned} &\text{maximize} && \textstyle\sum_{j=1}^{n} v_j x_j \\ &\text{subject to} && \textstyle\sum_{j=1}^{n} w_j x_j \leq W \\ &x_j \in \{0,1\} && \forall j = 1, \ldots, n \end{aligned}$$

where v_j is the value of item j, w_j is the weight of item j, W is the knapsack capacity, and $x_j = 1$ indicates item j is selected.

1.2 The Multidimensional Knapsack Problem

The MKP generalizes the classical knapsack problem to m constraints. Each item j now has m weights — one per constraint. Formally:

$$\begin{aligned} &\text{maximize} && \textstyle\sum_{j=1}^{n} v_j x_j \\ &\text{subject to} && \textstyle\sum_{j=1}^{n} w_{ji} x_j \leq W_i \quad \forall i = 1, \ldots, m \\ &x_j \in \{0,1\} && \forall j = 1, \ldots, n \end{aligned}$$

where w_{ji} is the weight of item j under constraint i and W_i is the capacity of constraint i. The classical knapsack is the special case where $m = 1$.

The MKP is NP-hard and is commonly solved using metaheuristics such as genetic algorithms (GAs). The most widely cited GA for the MKP is that of Chu and Beasley [**?**], which uses uniform random initialization — each bit is set to 1 with probability 0.5 independently of the problem structure. This approach ignores the feasibility landscape of the search space entirely, producing initial populations with low feasibility rates and poor solution quality, particularly on tight instances where capacity constraints are severely binding.

2 Proposed Idea

We propose using combinatorial generating functions to analytically characterize the feasibility structure of the MKP search space at the schema level before any search begins. For each item j, we compute a sub-generating function over the remaining items and use it to derive a feasibility density — the fraction of solutions that remain feasible when item j is included or excluded. These densities are used to set a per-bit sampling probability p_j for each item, biasing initialization toward high-feasibility regions of the search space.

This approach is purely combinatorial — it requires no optimization subproblem, no LP relaxation, and no prior search. It operates entirely on the problem data (item weights and capacities) and adds a one-time precomputation overhead before the GA begins. All other components of the GA — selection, crossover, mutation, and repair — remain identical to Chu and Beasley, isolating initialization as the sole variable under investigation.

3 Background

3.1 The Multidimensional Knapsack Problem

The MKP generalizes the classical 0-1 knapsack problem to m constraints. It is a well-studied NP-hard problem with applications in resource allocation, project selection, and capital budgeting. Benchmark instances are available from the OR-Library [**?**], which provides instances with up to $n = 500$ items and $m = 30$ constraints at varying tightness ratios $\alpha = W_i / \sum_j w_{ji}$.

3.2 Chu and Beasley Genetic Algorithm

Chu and Beasley [**?**] proposed a steady-state GA for the MKP that remains a standard baseline. Key components include uniform random initialization (each bit set to 1 with probability 0.5), uniform crossover, bit-flip mutation at rate $1/n$, binary tournament selection, and a deterministic repair operator that fixes infeasible solutions using a fixed LP-approximated efficiency ranking:

$$u_j = \frac{v_j}{\sum_{i=1}^{m} w_{ji}/W_i}$$

Because this ranking is computed once and never changes, diverse infeasible offspring are consistently repaired toward similar feasible solutions. Over successive generations, selection pressure favors high-efficiency items and the population converges prematurely — a well-documented weakness of the approach.

3.3 Schema Theory

Schema theory, introduced by Holland [**?**], provides a framework for analyzing the behavior of GAs at the level of partial solutions. A schema is a string of length n over the alphabet $\{0, 1, *\}$, where $*$ denotes a wildcard position that matches either 0 or 1. The order of a schema is the number of fixed positions. Schema theory suggests that short, low-order, above-average schemata are sampled exponentially more frequently over successive generations — a result known as the Schema Theorem.

3.4 Generating Functions in Combinatorics

A generating function is a formal power series whose coefficients encode combinatorial information. For a set of items with weights $w_1, \ldots, w_n$, the generating function:

$$f(x) = \prod_{j=1}^{n}(1 + x^{w_j})$$

has the property that the coefficient of x^k equals the number of subsets with total weight exactly k. This follows directly from the distributive law — each factor contributes either 1 (item excluded) or x^{w_j} (item included), and the product enumerates all 2^n subsets. Generating functions have been used extensively in partition theory and combinatorics [**?**] but have not, to our knowledge, been applied to bias GA initialization for the MKP.

4 Mathematical Framework

4.1 The Generating Function

Let there be n items and m constraints. Each item j has a weight w_{ji} for constraint i, where w_{ji} denotes the weight of item j under constraint i. Each constraint i has capacity W_i. A solution is a binary string $\mathbf{x} \in \{0,1\}^n$ where $x_j = 1$ indicates item j is selected. A solution is feasible if:

$$\sum_{j=1}^{n} w_{ji} x_j \leq W_i \quad \forall i = 1, \ldots, m$$

For a single constraint, the generating function over all 2^n subsets is:

$$f(x) = \prod_{j=1}^{n}(1 + x^{w_j}) = (1 + x^{w_1})(1 + x^{w_2}) \cdots (1 + x^{w_n})$$

The coefficient of x^k in $f(x)$ counts the number of subsets with total weight exactly k. Feasible subsets are those whose corresponding term has exponent $\leq W$.

For m constraints, this generalizes to m variables:

$$f(x_1, \ldots, x_m) = \prod_{j=1}^{n} \left(1 + x_1^{w_{j1}} x_2^{w_{j2}} \cdots x_m^{w_{jm}}\right)$$
$$= (1 + x_1^{w_{11}} x_2^{w_{12}} \cdots x_m^{w_{1m}})\,(1 + x_1^{w_{21}} x_2^{w_{22}} \cdots x_m^{w_{2m}}) \cdots (1 + x_1^{w_{n1}} x_2^{w_{n2}} \cdots x_m^{w_{nm}})$$

where w_{ji} is the weight of item j under constraint i. A term $x_1^{k_1} x_2^{k_2} \cdots x_m^{k_m}$ corresponds to a feasible subset if and only if $k_i \leq W_i$ for all $i = 1, \ldots, m$.

4.2 Sub-Generating Functions and Feasibility Density

To analyze the feasibility structure at the schema level, we define the *sub-generating function* for item j as the generating function over all remaining items with item j fixed or removed.

For a single constraint:

$$g_j(x) = \prod_{k \neq j}(1 + x^{w_k})$$
$$= (1 + x^{w_1}) \cdots (1 + x^{w_{j-1}})(1 + x^{w_{j+1}}) \cdots (1 + x^{w_n})$$

For m constraints:

$$g_j(x_1, \ldots, x_m) = \prod_{k \neq j} (1 + x_1^{w_{k1}} x_2^{w_{k2}} \cdots x_m^{w_{km}})$$

$$= (1 + x_1^{w_{11}} \cdots x_m^{w_{1m}}) \cdots \left(1 + x_1^{w_{(j-1)1}} \cdots x_m^{w_{(j-1)m}}\right) \left(1 + x_1^{w_{(j+1)1}} \cdots x_m^{w_{(j+1)m}}\right) \cdots (1 + x_1^{w_{n1}} \cdots x_m^{w_{nm}})$$

Note that g_j is identical whether item j is fixed to 0 or 1 — the difference lies in the capacity threshold applied.

We define the *feasibility densities* for item j as follows.

For a single constraint:

$$\rho_j^1 = \frac{\#\{\text{terms in } g_j(x) \text{ with exponent} \leq W - w_j\}}{2^{n-1}}$$

$$\rho_j^0 = \frac{\#\{\text{terms in } g_j(x) \text{ with exponent} \leq W\}}{2^{n-1}}$$

For m constraints:

$$\rho_j^1 = \frac{\#\{\text{terms in } g_j(x_1, \ldots, x_m) \text{ where exponent of } x_i \leq W_i - w_{ji}\ \forall i\}}{2^{n-1}}$$

$$\rho_j^0 = \frac{\#\{\text{terms in } g_j(x_1, \ldots, x_m) \text{ where exponent of } x_i \leq W_i\ \forall i\}}{2^{n-1}}$$

Intuitively, ρ_j^1 measures what fraction of solutions that include item j are feasible, and ρ_j^0 measures the same for solutions that exclude item j.

The sampling probability p_j is derived by normalizing ρ_j^1 against the total feasibility signal for item j. For any item j there are exactly two mutually exclusive choices — include it or exclude it — each with an associated feasibility density. We allocate probability to each choice proportionally to how feasibility-preserving that choice is, following a simple proportional allocation rule: if two options have scores a and b, the probability assigned to option a is:

$$P(a) = \frac{a}{a + b}$$

This ensures two properties hold simultaneously:

1. **The probabilities sum to 1:**

$$P(\text{include}) + P(\text{exclude}) = \frac{\rho_j^1}{\rho_j^1 + \rho_j^0} + \frac{\rho_j^0}{\rho_j^1 + \rho_j^0} = 1$$

2. **The ratio between probabilities equals the ratio between densities:** if excluding item j is twice as feasibility-preserving as including it, the algorithm is twice as likely to exclude it during initialization.

Note that we only report p_j — the probability of inclusion — since $P(\text{exclude}) = 1 - p_j$ follows automatically.

4.3 Order-1 Schemas

A schema is a string of length n over $\{0, 1, *\}$ where $*$ denotes a wildcard — a position that can take either value. The *order* of a schema is the number of fixed positions. For example with $n = 4$:

```
1*0*
```

is an order-2 schema — item 1 fixed to included, item 3 fixed to excluded, items 2 and 4 unconstrained.

An **order-1 schema** has exactly one fixed position and $n-1$ wildcards. For n items there are exactly $2n$ order-1 schemas — two per item:

- `*...1...*` — item j fixed to included
- `*...0...*` — item j fixed to excluded

Each order-1 schema contains exactly 2^{n-1} strings — all combinations of the remaining wildcard positions. The sub-generating function $g_j(x)$ defined in Section 4.2 corresponds exactly to the order-1 schema for item j, enumerating and characterizing the feasibility of all 2^{n-1} strings in that schema analytically.

We restrict our analysis to order-1 schemas because they provide exactly the per-item feasibility signal needed to compute p_j, at a cost of only $2n$ sub-generating function computations rather than the 3^n required for full schema enumeration. Note that in practice this reduces further to n computations — since $g_j(x)$ (or $g_j(x_1, \ldots, x_m)$ for m constraints) is identical for both the 0 and 1 fixing of item j, each sub-generating function is computed once and evaluated against two different capacity thresholds: $W - w_j$ for the included case and W for the excluded case (or $W_i - w_{ji}$ and W_i for all $i = 1, \ldots, m$ in the multi-constraint setting).

4.4 Sampling Probability

The per-bit sampling probability for item j is defined as:

$$p_j = \frac{\rho_j^1}{\rho_j^1 + \rho_j^0}$$

This is the probability of setting bit j to 1 when generating an initial solution. When $\rho_j^1 \ll \rho_j^0$, including item j severely restricts feasibility and p_j is pushed well below 0.5. When both densities are equal, $p_j = 0.5$, recovering the uniform initialization of Chu & Beasley.

5 Algorithm

Algorithm 1 Generating Function-Informed Initialization

Require: n items, m constraints, weights w_{ji}, capacities W_i, population size P
Ensure: Initial population of P solutions

```
Phase 1 — Precomputation (done once)
for j = 1 to n do
    Compute g_j(x_1, ..., x_m) = ∏_{k≠j} (1 + x_1^{w_k1} x_2^{w_k2} ··· x_m^{w_km})
    ρ_j^1 ← fraction of terms in g_j(x_1, ..., x_m) where exponent of x_i ≤ W_i − w_ji for all i = 1, ..., m
    ρ_j^0 ← fraction of terms in g_j(x_1, ..., x_m) where exponent of x_i ≤ W_i for all i = 1, ..., m
    p_j ← ρ_j^1 / (ρ_j^1 + ρ_j^0)
end for
return probability vector p = (p_1, p_2, ..., p_n)
Phase 2 — Population Initialization
for s = 1 to P do
    for j = 1 to n do
        Draw u ~ Uniform(0, 1)
        if u < p_j then
            x_j ← 1
        else
            x_j ← 0
        end if
    end for
    Add solution x to population
end for
return population
```

Note that Phase 1 requires exactly n sub-generating function computations — $g_j(x_1, \ldots, x_m)$ is computed once per item and evaluated against two capacity thresholds. Phase 2 is identical to the initialization of Chu & Beasley [**?**] except each bit j is sampled with its own problem-informed probability p_j rather than a uniform 0.5.

6 Worked Examples

6.1 Single Constraint Example

Consider the following MKP instance with $n = 3$ items and $m = 1$ constraint:

	Item 1	Item 2	Item 3
Weight	3	2	4
Value	5	4	3

Table 1: 3-item single constraint instance. Capacity $W = 5$.

Step 1 — Full Generating Function

$$f(x) = \prod_{j=1}^{3}(1 + x^{w_j}) = (1 + x^3)(1 + x^2)(1 + x^4)$$
$$= 1 + x^2 + x^3 + x^4 + x^5 + x^6 + x^7 + x^9$$

The 8 terms correspond to all $2^3 = 8$ subsets. Of these, 5 terms have exponent $\leq W = 5$, giving an overall feasibility density of $5/8 = 0.625$.

Step 2 — Sub-Generating Functions and Feasibility Densities

Each sub-generating function $g_j(x)$ is computed by removing item j's factor from the full product and evaluated against two thresholds: $W - w_j$ (included case) and W (excluded case). Each $g_j(x)$ has $2^{n-1} = 4$ terms.

Item 1 ($w_1 = 3,\ W - w_1 = 2$)*:*

$$g_1(x) = (1 + x^2)(1 + x^4) = 1 + x^2 + x^4 + x^6$$

Of 4 terms: 2 have exponent ≤ 2 and 3 have exponent ≤ 5.

$$\rho_1^1 = 2/4 = 0.50 \qquad \rho_1^0 = 3/4 = 0.75 \qquad p_1 = \frac{0.50}{0.50 + 0.75} = 0.40$$

Item 2 ($w_2 = 2,\ W - w_2 = 3$)*:*

$$g_2(x) = (1 + x^3)(1 + x^4) = 1 + x^3 + x^4 + x^7$$

Of 4 terms: 2 have exponent ≤ 3 and 3 have exponent ≤ 5.

$$\rho_2^1 = 2/4 = 0.50 \qquad \rho_2^0 = 3/4 = 0.75 \qquad p_2 = \frac{0.50}{0.50 + 0.75} = 0.40$$

Item 3 ($w_3 = 4,\ W - w_3 = 1$)*:*

$$g_3(x) = (1 + x^3)(1 + x^2) = 1 + x^2 + x^3 + x^5$$

Of 4 terms: 1 has exponent ≤ 1 and 4 have exponent ≤ 5.

$$\rho_3^1 = 1/4 = 0.25 \qquad \rho_3^0 = 4/4 = 1.00 \qquad p_3 = \frac{0.25}{0.25 + 1.00} = 0.20$$

Step 3 — Summary, Comparison, and Expected Feasibility Rate

The table below summarises the feasibility densities and resulting sampling probabilities for all three items, compared against the uniform baseline of Chu & Beasley.

	Item 1	Item 2	Item 3
Weight	3	2	4
ρ_j^1	0.50	0.50	0.25
ρ_j^0	0.75	0.75	1.00
p_j **(ours)**	**0.40**	**0.40**	**0.20**
p_j (Chu & Beasley)	0.50	0.50	0.50

Table 2: Per-item feasibility densities and sampling probabilities. $p_j = \rho_j^1/(\rho_j^1 + \rho_j^0)$ as defined in Section 4.4.

Item 3 is the heaviest item — weight 4 out of capacity 5, consuming 80% of the knapsack upon inclusion. The generating function captures this automatically, pushing p_3 down to 0.20. To confirm that these adjusted probabilities translate into a measurably higher fraction of feasible initial solutions, we compute the expected feasibility rate directly.

Since bits are sampled independently, the probability of generating a specific string $\mathbf{x} = (x_1, x_2, x_3)$ under our method is:

$$P(\mathbf{x}) = \prod_{j=1}^{3} \left[p_j^{x_j} \cdot (1 - p_j)^{1-x_j} \right]$$

For each bit position j, we multiply p_j if $x_j = 1$ and $(1 - p_j)$ if $x_j = 0$. The expected feasibility rate is the sum of $P(\mathbf{x})$ over all feasible strings — those with total weight $\leq W = 5$.

String	Weight	Feasible?	$P(\mathbf{x} \mid \text{ours})$	$P(\mathbf{x} \mid \text{C\&B})$
000	0	Yes	$0.60 \times 0.60 \times 0.80 = 0.2880$	0.1250
001	4	Yes	$0.60 \times 0.60 \times 0.20 = 0.0720$	0.1250
010	2	Yes	$0.60 \times 0.40 \times 0.80 = 0.1920$	0.1250
011	6	No	$0.60 \times 0.40 \times 0.20 = 0.0480$	0.1250
100	3	Yes	$0.40 \times 0.60 \times 0.80 = 0.1920$	0.1250
101	7	No	$0.40 \times 0.60 \times 0.20 = 0.0480$	0.1250
110	5	Yes	$0.40 \times 0.40 \times 0.80 = 0.1280$	0.1250
111	9	No	$0.40 \times 0.40 \times 0.20 = 0.0320$	0.1250
Expected feasibility rate			**0.8720**	**0.6250**

Table 3: String-level probability breakdown for the single constraint instance ($W = 5$). Under Chu and Beasley all strings are equally likely at $1/8 = 0.1250$. Under our method probability mass shifts toward feasible strings, improving the expected feasibility rate from 62.5% to 87.2% — a gain of 24.7 percentage points.

Item 3 is the primary driver of this improvement. By reducing p_3 from 0.50 to 0.20, probability mass shifts away from the three infeasible strings that all contain item 3 (strings 011, 101, and 111). Chu and Beasley sample infeasible solutions 37.5% of the time; our method reduces this to 12.8%.

6.2 Multiple Constraint Example

Consider the following MKP instance with $n = 4$ items and $m = 2$ constraints:

	Item 1	Item 2	Item 3	Item 4
Weight (C1)	2	4	3	5
Weight (C2)	3	2	5	1
Value	4	7	5	3

Table 4: 4-item two-constraint instance. Capacities $W_1 = 7$, $W_2 = 8$.

Step 1 — Full Generating Function

$$f(x_1, x_2) = \prod_{j=1}^{4} \left(1 + x_1^{w_{j1}} x_2^{w_{j2}}\right)$$
$$= (1 + x_1^2 x_2^3)(1 + x_1^4 x_2^2)(1 + x_1^3 x_2^5)(1 + x_1^5 x_2^1)$$

Each of the $2^4 = 16$ terms corresponds to one subset. A term $x_1^{k_1} x_2^{k_2}$ is feasible if and only if $k_1 \leq W_1 = 7$ and $k_2 \leq W_2 = 8$ simultaneously. Of the 16 subsets, 9 are feasible, giving an overall feasibility density of $9/16 = 0.5625$.

Step 2 — Sub-Generating Functions and Feasibility Densities

For each item j we compute $g_j(x_1, x_2)$ by removing item j's factor from the full product. Each $g_j(x_1, x_2)$ is evaluated against two capacity thresholds simultaneously — $W_i - w_{ji}$ when item j is included and W_i when excluded — for all $i = 1, \ldots, m$. Each sub-generating function has $2^{n-1} = 8$ terms.

Item 1 ($w_{11} = 2,\ w_{12} = 3,\ W_1 - w_{11} = 5,\ W_2 - w_{12} = 5$)*:*

$$g_1(x_1, x_2) = (1 + x_1^4 x_2^2)(1 + x_1^3 x_2^5)(1 + x_1^5 x_2^1)$$

Items included	$(k_1,\ k_2)$	$k_1 \leq 5,\ k_2 \leq 5$	$k_1 \leq 7,\ k_2 \leq 8$
none	(0, 0)	✓	✓
item 2 only	(4, 2)	✓	✓
item 3 only	(3, 5)	✓	✓
item 4 only	(5, 1)	✓	✓
items 2, 3	(7, 7)	✗	✓
items 2, 4	(9, 3)	✗	✗
items 3, 4	(8, 6)	✗	✗
items 2, 3, 4	(12, 8)	✗	✗

Table 5: Feasibility of all $2^3 = 8$ terms of $g_1(x_1, x_2)$ under both capacity thresholds. Left column applies when item 1 is included ($W_i - w_{1i}$); right column when excluded (W_i).

$$\rho_1^1 = 4/8 = 0.500 \qquad \rho_1^0 = 5/8 = 0.625 \qquad p_1 = \frac{0.500}{0.500 + 0.625} = 0.444$$

Item 2 ($w_{21} = 4,\ w_{22} = 2,\ W_1 - w_{21} = 3,\ W_2 - w_{22} = 6$)*:*

$g_2(x_1, x_2) = (1 + x_1^2 x_2^3)(1 + x_1^3 x_2^5)(1 + x_1^5 x_2^1)$

Items included	$(k_1,\ k_2)$	$k_1 \leq 3,\ k_2 \leq 6$	$k_1 \leq 7,\ k_2 \leq 8$
none	(0, 0)	✓	✓
item 1 only	(2, 3)	✓	✓
item 3 only	(3, 5)	✓	✓
item 4 only	(5, 1)	✗	✓
items 1, 3	(5, 8)	✗	✓
items 1, 4	(7, 4)	✗	✓
items 3, 4	(8, 6)	✗	✗
items 1, 3, 4	(10, 9)	✗	✗

Table 6: Feasibility of all $2^3 = 8$ terms of $g_2(x_1, x_2)$ under both capacity thresholds.

$$\rho_2^1 = 3/8 = 0.375 \qquad \rho_2^0 = 6/8 = 0.750 \qquad p_2 = \frac{0.375}{0.375 + 0.750} = 0.333$$

Item 3 ($w_{31} = 3,\ w_{32} = 5,\ W_1 - w_{31} = 4,\ W_2 - w_{32} = 3$)*:*

$g_3(x_1, x_2) = (1 + x_1^2 x_2^3)(1 + x_1^4 x_2^2)(1 + x_1^5 x_2^1)$

Items included	$(k_1,\ k_2)$	$k_1 \leq 4,\ k_2 \leq 3$	$k_1 \leq 7,\ k_2 \leq 8$
none	(0, 0)	✓	✓
item 1 only	(2, 3)	✓	✓
item 2 only	(4, 2)	✓	✓
item 4 only	(5, 1)	✗	✓
items 1, 2	(6, 5)	✗	✓
items 1, 4	(7, 4)	✗	✓
items 2, 4	(9, 3)	✗	✗
items 1, 2, 4	(11, 7)	✗	✗

Table 7: Feasibility of all $2^3 = 8$ terms of $g_3(x_1, x_2)$ under both capacity thresholds.

$$\rho_3^1 = 3/8 = 0.375 \qquad \rho_3^0 = 6/8 = 0.750 \qquad p_3 = \frac{0.375}{0.375 + 0.750} = 0.333$$

Item 4 ($w_{41} = 5,\ w_{42} = 1,\ W_1 - w_{41} = 2,\ W_2 - w_{42} = 7$)*:*

$g_4(x_1, x_2) = (1 + x_1^2 x_2^3)(1 + x_1^4 x_2^2)(1 + x_1^3 x_2^5)$

Items included	$(k_1,\ k_2)$	$k_1 \le 2,\ k_2 \le 7$	$k_1 \le 7,\ k_2 \le 8$
none	(0, 0)	✓	✓
item 1 only	(2, 3)	✓	✓
item 2 only	(4, 2)	✗	✓
item 3 only	(3, 5)	✗	✓
items 1, 2	(6, 5)	✗	✓
items 1, 3	(5, 8)	✗	✓
items 2, 3	(7, 7)	✗	✓
items 1, 2, 3	(9, 10)	✗	✗

Table 8: Feasibility of all $2^3 = 8$ terms of $g_4(x_1, x_2)$ under both capacity thresholds.

$$\rho_4^1 = 2/8 = 0.250 \qquad \rho_4^0 = 7/8 = 0.875 \qquad p_4 = \frac{0.250}{0.250 + 0.875} = 0.222$$

Step 3 — Summary, Comparison, and Expected Feasibility Rate

The table below summarises the feasibility densities and resulting sampling probabilities for all four items.

	Item 1	Item 2	Item 3	Item 4
Weight (C1)	2	4	3	5
Weight (C2)	3	2	5	1
ρ_j^1	0.500	0.375	0.375	0.250
ρ_j^0	0.625	0.750	0.750	0.875
p_j **(ours)**	**0.444**	**0.333**	**0.333**	**0.222**
p_j (Chu & Beasley)	0.500	0.500	0.500	0.500

Table 9: Per-item feasibility densities and sampling probabilities for the two-constraint instance ($W_1 = 7$, $W_2 = 8$). $p_j = \rho_j^1/(\rho_j^1 + \rho_j^0)$ as defined in Section 4.4.

Item 4 receives the most aggressive adjustment — its large weight on C1 ($w_{41} = 5$) restricts joint feasibility across both constraints simultaneously, pushing p_4 down to 0.222. Items 2 and 3 both receive $p_j = 0.333$ despite stressing different constraints: item 2 is heavy on C1, item 3 is heavy on C2, but both impose comparable joint feasibility costs. To confirm that these adjusted probabilities produce a measurably higher fraction of feasible initial solutions, we compute the expected feasibility rate directly.

The probability of generating a specific string $\mathbf{x} = (x_1, x_2, x_3, x_4)$ under our method is:

$$P(\mathbf{x}) = \prod_{j=1}^{4} \left[p_j^{x_j} \cdot (1 - p_j)^{1 - x_j} \right]$$

A string is feasible if and only if $k_1 \le W_1 = 7$ and $k_2 \le W_2 = 8$ simultaneously. We enumerate all 16 strings below, using the rounded values $1 - p_1 \approx 0.556$, $1 - p_2 = 1 - p_3 \approx 0.667$, $1 - p_4 \approx 0.778$.

String	$(k_1,\ k_2)$	Feasible?	$P(\mathbf{x} \mid \text{ours})$	$P(\mathbf{x} \mid \text{C\&B})$
0000	(0, 0)	Yes	$0.556 \times 0.667 \times 0.667 \times 0.778 = 0.1928$	0.0625
0001	(5, 1)	Yes	$0.556 \times 0.667 \times 0.667 \times 0.222 = 0.0550$	0.0625
0010	(3, 5)	Yes	$0.556 \times 0.667 \times 0.333 \times 0.778 = 0.0963$	0.0625
0011	(8, 6)	No	$0.556 \times 0.667 \times 0.333 \times 0.222 = 0.0275$	0.0625
0100	(4, 2)	Yes	$0.556 \times 0.333 \times 0.667 \times 0.778 = 0.0963$	0.0625
0101	(9, 3)	No	$0.556 \times 0.333 \times 0.667 \times 0.222 = 0.0275$	0.0625
0110	(7, 7)	Yes	$0.556 \times 0.333 \times 0.333 \times 0.778 = 0.0481$	0.0625
0111	(12, 8)	No	$0.556 \times 0.333 \times 0.333 \times 0.222 = 0.0137$	0.0625
1000	(2, 3)	Yes	$0.444 \times 0.667 \times 0.667 \times 0.778 = 0.1540$	0.0625
1001	(7, 4)	Yes	$0.444 \times 0.667 \times 0.667 \times 0.222 = 0.0440$	0.0625
1010	(5, 8)	Yes	$0.444 \times 0.667 \times 0.333 \times 0.778 = 0.0770$	0.0625
1011	(10, 9)	No	$0.444 \times 0.667 \times 0.333 \times 0.222 = 0.0220$	0.0625
1100	(6, 5)	Yes	$0.444 \times 0.333 \times 0.667 \times 0.778 = 0.0770$	0.0625
1101	(11, 6)	No	$0.444 \times 0.333 \times 0.667 \times 0.222 = 0.0220$	0.0625
1110	(9, 10)	No	$0.444 \times 0.333 \times 0.333 \times 0.778 = 0.0385$	0.0625
1111	(14, 11)	No	$0.444 \times 0.333 \times 0.333 \times 0.222 = 0.0110$	0.0625
Expected feasibility rate			**0.7405**	**0.5625**

Table 10: String-level probability breakdown for the two-constraint instance ($W_1 = 7$, $W_2 = 8$). Of the 16 possible strings, 9 are feasible. Under Chu and Beasley the expected feasibility rate is $9/16 = 0.5625$. Under our method probability mass shifts toward feasible strings, improving the expected feasibility rate to 74.1% — a gain of 17.8 percentage points.

Item 4 is the primary driver of this improvement. By reducing p_4 from 0.500 to 0.222, our method assigns substantially lower probability mass to strings containing item 4, which are disproportionately infeasible due to its large weight on C1. Chu and Beasley sample infeasible solutions 43.75% of the time; our method reduces this to 25.95%.

7 Monte Carlo Estimation of Feasibility Densities

7.1 Motivation

The worked examples in Section 6 compute ρ_j^1 and ρ_j^0 exactly by expanding the sub-generating function g_j and enumerating all 2^{n-1} terms. For $n = 3$ and $n = 4$ this is tractable. For real MKP instances with $n = 100$ items, exact enumeration requires examining 2^{99} subsets per item — completely infeasible. We therefore approximate these quantities via Monte Carlo sampling.

7.2 What We Are Estimating

For each item j we require two quantities:

- ρ_j^1 — the fraction of all 2^{n-1} subsets of the remaining items that remain feasible when item j is included, checked against the reduced capacity $W_i - w_{ji}$ for all constraints i simultaneously.
- ρ_j^0 — the same fraction when item j is excluded, checked against the full capacity W_i.

The sampling probability is then $p_j = \rho_j^1/(\rho_j^1 + \rho_j^0)$ as defined in Section 4.4.

7.3 The Estimation Procedure

Instead of enumerating all 2^{n-1} subsets, we draw M random binary strings of length n uniformly at random. For each string and each item j we compute the total constraint usage of the string under all m constraints, remove item j's contribution from the usage, and check feasibility against $W_i - w_{ji}$ (included case) and W_i (excluded case) for all i simultaneously. A string contributes to ρ_j^1 only when $x_j = 1$ and to ρ_j^0 only when $x_j = 0$ — each sample therefore contributes to exactly one of the two counts, ensuring that each estimator averages over strings that actually fix item j to the corresponding value rather than mixing both cases. Feasible counts are accumulated for each item across all M samples, and the empirical fractions yield estimates of ρ_j^1 and ρ_j^0.

A critical property of this procedure is that Monte Carlo estimation is performed exactly once per problem instance. The probability vector $\mathbf{p} = (p_1, p_2, \ldots, p_n)$ is computed once before any GA run begins and reused across all subsequent runs on that instance. The computational cost is entirely front-loaded and independent of the number of GA runs performed.

7.4 Statistical Justification

The true ρ_j^1 is the probability that a uniformly random subset of the remaining $n-1$ items is feasible under the reduced capacity. The empirical fraction over M samples is an unbiased estimator of this probability. By the law of large numbers the estimate converges to the true value as M grows. The standard error of the estimate is bounded by:

$$\sqrt{\frac{p(1-p)}{M}} \leq \frac{1}{2\sqrt{M}}$$

At $M = 10{,}000{,}000$ samples this gives a maximum standard error of approximately 0.016% — effectively exact for the purpose of setting initialization probabilities. A deviation of this magnitude in p_j has negligible impact on GA behavior since p_j is used only for probabilistic sampling during initialization.

7.5 Connection to the Worked Examples

The worked examples in Section 6 compute ρ_j^1 and ρ_j^0 exactly by expanding g_j and counting feasible terms. Monte Carlo approximates exactly the same quantities by random sampling instead of enumeration. For $n = 3$ and $n = 4$ both methods yield identical results. For $n = 100$ only Monte Carlo is computationally feasible.

7.6 GPU Implementation

The naive implementation loops over M samples and n items — approximately $M \times n = 10^9$ operations for $M = 10{,}000{,}000$ and $n = 100$. On CPU this would require hours. We vectorize the entire computation using PyTorch tensors on a GPU. All M binary strings are generated simultaneously as an $(M \times n)$ tensor, and constraint usage for all strings is computed at once via matrix multiplication:

$$\mathbf{U} = \mathbf{X}\mathbf{W}^\top \quad \in \mathbb{R}^{M \times m}$$

where $\mathbf{X} \in \{0,1\}^{M \times n}$ is the sample matrix and $\mathbf{W} \in \mathbb{R}^{m \times n}$ is the weight matrix. For each item j, its contribution is subtracted and feasibility is checked against both capacity thresholds in parallel across all M strings using tensor operations and GPU-accelerated reductions. This reduces wall time from hours to approximately 7 seconds on an NVIDIA T4 GPU, making the precomputation practical for all standard OR-Library benchmark instances.

Algorithm 2 Monte Carlo Feasibility Density Estimation (GPU-Vectorized)

Require: n items, m constraints, weights $\mathbf{W} \in \mathbb{R}^{m \times n}$, capacities $\mathbf{c} \in \mathbb{R}^m$, sample count M, batch size B

Ensure: Probability vector $\mathbf{p} = (p_1, \ldots, p_n)$

for each batch of B random binary strings $\mathbf{X} \in \{0,1\}^{B \times n}$ **do**
  $\mathbf{U} \leftarrow \mathbf{X}\mathbf{W}^\top$ ▷ constraint usage, shape $(B \times m)$
  for $j = 1$ to n **do**
    $\mathbf{U}_{-j} \leftarrow \mathbf{U} - \mathbf{X}_{:,j} \cdot \mathbf{W}_{:,j}^\top$ ▷ remove item j's contribution
    $\text{feas_in}_j \leftarrow (\mathbf{U}_{-j} \leq \mathbf{c} - \mathbf{W}_{:,j}).\text{all}(\text{dim} = 1) \ \wedge \ \mathbf{X}_{:,j} = 1$
    $\text{feas_out}_j \leftarrow (\mathbf{U}_{-j} \leq \mathbf{c}).\text{all}(\text{dim} = 1) \ \wedge \ \mathbf{X}_{:,j} = 0$
    Accumulate counts into count_in_j, count_out_j, total_in_j, total_out_j
  end for
end for
for $j = 1$ to n **do**
  $\rho_j^1 \leftarrow \text{count_in}_j \ / \ \text{total_in}_j$
  $\rho_j^0 \leftarrow \text{count_out}_j \ / \ \text{total_out}_j$
  $p_j \leftarrow \rho_j^1 \ / \ (\rho_j^1 + \rho_j^0)$
end for
return p